

Foundations of AI(CS 5100)

Automated Stock Trading - RL and LSTM

TEAM:

KAVINN PREMMESH TAMILARASU

PRAKASH KUMAR RAMESH

SARVESH ELANCHEZHIAN

Introduction.....	3
Methods.....	5
Overview.....	5
Data Collection and Preprocessing.....	5
Approach 1: LSTM Price Forecasting with Heuristic Trading.....	5
Model Architecture.....	6
Training Procedure.....	6
Heuristic Trading Strategy.....	8
Approach 2: Reinforcement Learning with PPO.....	8
Problem Formulation:.....	8
Data and Feature Engineering:.....	8
Custom Trading Environment:.....	9
PPO Agent Architecture:.....	10
Training Procedure:.....	10
Evaluation Framework.....	11
External Software and Data Sources.....	11
Results.....	12
Conclusion.....	14
Strengths.....	14
Weaknesses.....	15
Comparison with Related Work.....	16
Future Improvements.....	16
Summary.....	17
Works Cited.....	17

Introduction

Can machines reliably make money in the stock market? That question has fueled decades of work across economics, statistics, and more recently, artificial intelligence. Markets are some of the most data-rich, high-stakes environments around for sequential decision-making, but they are also notoriously hard to model. Prices shift constantly, driven by a messy combination of quantifiable signals and human emotions that resist measurement. Worse, any pattern that traders catch on to tends to vanish as people rush to exploit it. That mix of complexity, noise, and adversarial behavior makes automated trading one of the toughest proving grounds for machine learning.

The earliest algorithmic trading systems were rule-based. They would mechanically fire off trades based on preset strategies like moving-average crossovers, momentum filters, or mean-reversion bands pulled from classical technical analysis. These methods still see use today, but their dependence on hand-tuned thresholds makes them brittle when market conditions change. As machine learning took off in the 2010s, a wave of data-driven alternatives followed. Supervised learning approaches, especially recurrent neural networks and gated variants like Long Short-Term Memory (LSTM) networks, reframed the problem as time-series regression. The idea was simple - feed the model a window of historical prices and volume data along with some technical indicators, have it predict tomorrow's closing price, and then turn that prediction into a trading signal using a threshold rule. These models showed that raw price sequences do contain learnable statistical structure, and LSTM-based predictors have reported competitive forecasting accuracy on individual stocks. But they have a fundamental problem. They are optimized for prediction accuracy, not trading profit. A model can nail its price forecasts on held-out data and still lose money when those predictions get turned into actual trades. The reason is that the mapping from a price forecast to an optimal action depends on the current state of the portfolio, transaction costs, and the compounding effects of decisions over time. A single-step regression loss captures none of that.

This gap between predicting and deciding is exactly what makes the reinforcement learning (RL) framing of trading so appealing. Rather than trying to forecast prices, an RL agent focuses on a different question: given the current market conditions and my portfolio, what trade should I make to grow my wealth over time? This is essentially what a Markov Decision Process formalizes, with a state space, action space, reward signal, and transition dynamic. The agent interacts with a simulated market, receives a reward tied to changes in its net worth at each step, and learns a policy mapping from states to actions that maximizes the expected discounted sum of future rewards. No labeled actions are needed. The agent figures out what works through trial and error, directly optimizing the thing we actually care about.

The work that most directly inspires this project is Yang et al. (2020), "Deep Reinforcement Learning for Automated Stock Trading: An Ensemble Strategy," published at the ACM International Conference on AI in Finance. Yang et al. introduced FinRL, an open-source framework that formalizes single-stock and multi-stock trading as MDPs and benchmarks three deep RL algorithms (Proximal Policy Optimization, or PPO; Advantage Actor-Critic, or A2C; and Deep Deterministic Policy Gradient, or DDPG) on a Dow Jones 30 portfolio. Their results are worth noting. PPO, an on-policy algorithm that uses a clipped surrogate objective to keep each policy update from getting too large and blowing up training, consistently delivered competitive risk-adjusted returns. They also showed that state representation matters a lot. Adding technical indicators like MACD, RSI, and moving averages on top of raw OHLCV data gives the agent compressed summaries of trend and momentum that make learning more stable. We adopt several of their design choices directly in our implementation, including their 0.1% per-trade transaction fee, their action formulation (all-in buy, all-out sell, hold), and their approach to normalizing rewards by initial account balance.

This project builds a self-contained automated trading system that puts three strategies head to head. A shared data pipeline pulls historical daily market data for five FAANG stocks (Meta, Apple, Amazon, Netflix, and Alphabet), enriches it with standard technical indicators, and splits it chronologically into training and test sets so there is no look-ahead bias. On top of that data, we implement and compare two learning-based approaches against a passive baseline. The first is an LSTM price predictor that treats trading as supervised regression, converting next-day forecasts into Buy, Sell, or Hold signals through a threshold rule. The second is a PPO agent trained inside a custom Gymnasium environment that treats portfolio management directly as a Markov Decision Process, learning a trading policy through interaction with the simulated market without ever explicitly predicting prices. All three strategies (LSTM, PPO, and Buy-and-Hold) are evaluated on a shared out-of-sample test window using cumulative return, annualized Sharpe ratio, and maximum drawdown.

Our experiments reveal a finding that is both practically important and consistent with the broader literature. In the strong bull market that defines our test period, passive Buy-and-Hold is genuinely hard to beat with any active strategy. The PPO agent does achieve positive returns across all five tickers, which shows it successfully learns a profitable policy, but its risk-averse tendencies hold it back in a sustained uptrend. The LSTM model runs into its own structural issue: strong price forecast accuracy does not automatically translate to strong trading performance when the market keeps moving in one direction. Taken together, these results highlight that the real value of active ML strategies shows up most clearly in volatile or mean-reverting markets, which has direct implications for when and how these systems should actually be used.

The rest of this report is organized as follows. The Methods section covers the data pipeline, feature engineering, train-test protocol, and the architecture and training procedures for both the LSTM and PPO models. The Results section presents the comparative evaluation across FAANG tickers, reporting cumulative return, Sharpe ratio, and maximum drawdown. The

Conclusion discusses key findings, limitations, and directions for future work. Full references are provided in the Works Cited section.

Methods

Overview

Our system tackles the automated stock trading problem through two complementary approaches. The first is a supervised learning approach built around a Long Short-Term Memory (LSTM) network that forecasts next-day closing prices, with trading decisions derived from a threshold-based heuristic applied to those forecasts. The second is a reinforcement learning approach that uses Proximal Policy Optimization (PPO) to learn a trading policy directly from interaction with a simulated market environment. Both strategies are benchmarked against a passive Buy-and-Hold baseline across the FAANG stocks (META, AAPL, AMZN, NFLX, GOOGL) using a shared evaluation framework that computes cumulative return, annualized Sharpe ratio, and maximum drawdown.

Data Collection and Preprocessing

Daily OHLCV (Open, High, Low, Close, Volume) data for each ticker was downloaded from Yahoo Finance[3] using the *yfinance* Python library, covering the period from January 1, 2015 through December 31, 2024. After downloading, we computed three families of technical indicators from the closing price series and appended them as additional features. Relative Strength Index (RSI) was computed with a 14-period exponentially weighted window. MACD was derived using the standard 12/26/9 configuration, yielding three columns: the MACD line, the signal line, and the histogram. Simple and exponential moving averages were computed at three horizons (7, 21, and 50 days), producing six additional columns. After dropping warm-up rows where any indicator was undefined, the final feature set comprised 15 columns per timestep.

The data for each ticker was split chronologically into 80% training and 20% test, with no shuffling, preserving the temporal ordering of the market. Normalization was performed using *MinMaxScaler* from scikit-learn, fit exclusively on the training partition to prevent data leakage. Price-related columns (OHLC, all moving averages, and the MACD family) shared a single scaler; Volume used a separate scaler; RSI was simply divided by 100. The fitted scalers were persisted via *joblib* so that the same transformation could be applied consistently during inference. Both raw (unscaled) and normalized CSVs were saved per ticker, as the raw prices are needed for portfolio simulation while the normalized features are consumed by the models.

Approach 1: LSTM Price Forecasting with Heuristic Trading

Model Architecture

The LSTM component is a supervised next-day closing price forecaster. We designed a multi-ticker architecture called *MultiTickerLSTMPricePredictor* that trains a single shared model across all five FAANG tickers simultaneously, rather than fitting separate models per stock.

At each timestep, the 15-dimensional normalized feature vector is concatenated with a 16-dimensional learned ticker embedding, producing a 31-dimensional input. The ticker embedding is an *nn.Embedding* layer with 5 entries (one per FAANG stock), and the same embedding vector is broadcast across all timesteps in the look-back window. This allows the network to learn both shared cross-stock dynamics and per-symbol characteristics within a common parameter space.

The concatenated input sequence, spanning a window of 45 days, is processed by a 2-layer stacked LSTM with a hidden dimension of 64 and inter-layer dropout of 0.3. Rather than relying solely on the final hidden state, we introduced a Bahdanau-style temporal attention mechanism. A learned linear projection scores each of the 45 timestep outputs, softmax-normalizes them into attention weights, and computes a weighted sum to form a single context vector. This enables the model to attend to the most informative days within the window, which is particularly useful since relevant signals (earnings surprises, regime changes) may appear at any position in the look-back period, not just the most recent day.

The context vector is passed through a fully connected head consisting of a linear layer (64 to 64), ReLU activation, dropout (0.3), and a final linear layer (64 to 1), producing a single scalar output: the predicted next-day normalized closing price.

Training Procedure

The model was trained using mean squared error (MSE) loss with the Adam optimizer. Within the 80% training partition, a further chronological 80/20 split was used for training and validation. Sliding window samples from all five tickers were pooled together into a single training set, with each sample carrying its integer ticker index for the embedding lookup.

We applied several regularization and optimization techniques to improve generalization. Early stopping halted training if validation loss did not improve for 15 consecutive epochs. Gradient clipping with a maximum norm of 1.0 prevented exploding gradients. L2 weight decay of $1e-4$ was applied through the Adam optimizer. A ReduceLROnPlateau scheduler halved the learning rate after 5 epochs of no validation improvement. The initial learning rate was set to $5e-4$, batch size to 64, and maximum number of epochs to 80, though early stopping typically triggered well before the limit.

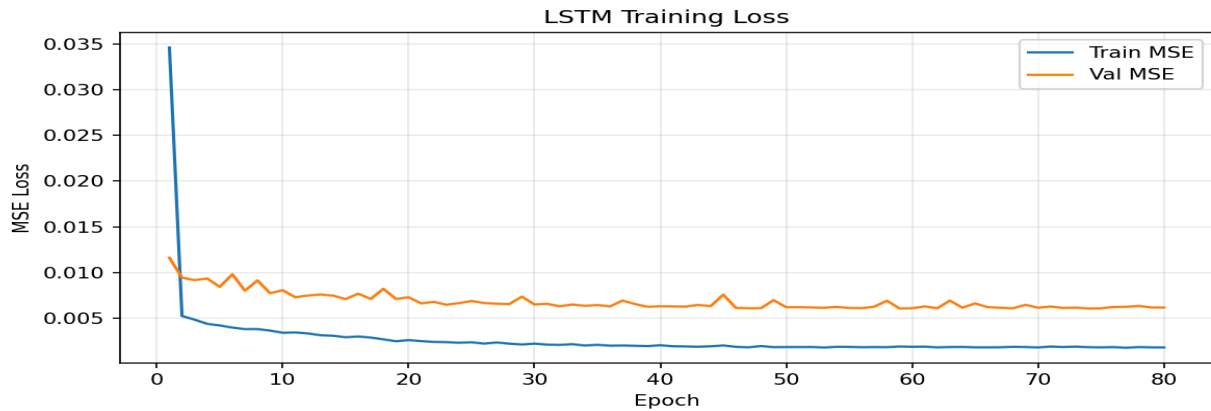


Figure 2.1 LSTM Training Loss

The training and validation MSE loss curves over the full 80 epochs. The training loss drops sharply within the first few epochs, falling from approximately 0.035 to below 0.005, and continues to decrease gradually before plateauing around 0.002. The validation loss follows a similar trend but stabilizes higher, around 0.006, with minor fluctuations throughout training. The persistent gap between the two curves suggests a modest degree of overfitting, though the validation loss remains stable and does not increase, indicating that the model generalizes reasonably well without severe degradation on unseen data.

Hyperparameter	Value
Window size	45 days
Hidden dimension	64
LSTM layers	2
Embedding dimension	16
Dropout	0.3
Learning rate	5×10^{-4}
Weight decay (L2)	1×10^{-4}
Gradient clipping	1.0
Batch size	64
Max epochs	80
Early stopping	15 epochs patience
LR scheduler	ReduceLROnPlateau (factor=0.5, patience=5)
Loss function	MSE
Optimizer	Adam

Figure 2.2 Hyperparameters Used in LSTM

The above table summarizes the hyperparameters used for the LSTM model. The network uses a look-back window of 45 days, processing the input through 2 stacked LSTM layers with a hidden dimension of 64 and a dropout rate of 0.3. Each ticker is represented by a 16-dimensional learned embedding. Training was conducted using the Adam optimizer with MSE loss, an initial learning rate of 5×10^{-4} , L2 weight decay of 1×10^{-4} , and a batch size of 64. Gradient clipping was set at a maximum norm of 1.0 to prevent exploding gradients. The model was allowed to train for up to 80 epochs, with early stopping at a patience of 15 epochs to avoid overfitting. A ReduceLROnPlateau scheduler further regulated training by halving the learning rate after 5 epochs of no validation improvement.

Heuristic Trading Strategy

Since the LSTM is a price forecaster and does not inherently produce trading decisions, we wrapped it with a threshold-based heuristic to convert predictions into a long-only trading strategy. On each test day, the model produces a predicted next-day closing price in dollar terms (after inverse-scaling the normalized output). This prediction is compared against a reference price, defined as the previous day's actual closing price. If the prediction exceeds the reference by more than a threshold fraction, the strategy buys (investing all available cash at the current price). If the prediction falls below the reference by more than the threshold, the strategy sells (liquidating all shares). If the prediction lies within the dead-zone band, no action is taken.

A 0.1% transaction fee is applied on both buys and sells, matching the cost model used in the reinforcement learning environment for fair comparison. The threshold was auto-selected per ticker from a grid of candidates (0.5%, 0.3%, 0.2%, 0.1%, 0.05%) by choosing the value that maximized final portfolio value.

Approach 2: Reinforcement Learning with PPO

Problem Formulation:

The stock trading problem is formulated as a discrete-time Markov Decision Process, defined by the tuple (S, A, R, T) , where S is the state space, A is the action space, R is the reward function, and T is the transition dynamic governed by the market. At each trading day t , the agent observes a state s_t , selects an action a_t , and receives a scalar reward r_t reflecting the immediate consequence of that action on the portfolio. The agent's objective is to learn a policy $p: S \rightarrow A$ that maximizes the expected discounted cumulative reward $E[\sum(\gamma^t r_t)]$, where γ is the discount factor. This formulation is adopted directly from [1], whose FinRL framework established it as the standard paradigm for single-asset deep RL trading.

Data and Feature Engineering:

Historical daily market data for FAANG equities were retrieved programmatically from

Yahoo Finance using the `yfinance` library, covering the period from January 2015 to December 2024, yielding 2,515 trading days per ticker. The raw OHLCV columns (Open, High, Low, Close, Volume) were enriched with seven technical indicators computed per ticker:

- Relative Strength Index (RSI), 14-period: measures momentum and overbought/oversold conditions.
- MACD (12/26/9 config): captures trend direction and momentum through the difference of two exponential moving averages, along with its signal line and histogram.
- Simple Moving Averages at 7, 21, and 50-day windows (SMA_7, SMA_21, SMA_50).
- Exponential Moving Averages at 7, 21, and 50-day windows (EMA_7, EMA_21, EMA_50).

These 15 features form the market observation vector. A 80/20 train-test split (1,972 training days, 494 test days per ticker) was applied with no shuffling, ensuring the test period simulates genuine out-of-sample trading and preventing any look-ahead bias. Min-max normalization was fit exclusively on the training split and applied to both splits, with fitted scaler objects saved per ticker to support inverse transforms during evaluation. Critically, two versions of each split are maintained. A normalized version used for observations and a raw-price version used for trade execution, preventing the agent from treating normalized prices as dollar amounts.

Custom Trading Environment:

The trading environment was implemented as a custom class conforming to the Gymnasium API [4], the standard interface for RL environment development. Each episode corresponds to one pass through the full training dataset, with the agent making one decision per trading day.

State space. The observation at each step is an 18-dimensional vector. The 15 normalized market features concatenated with three live portfolio state values - current cash balance, shares held, and total net worth. This design ensures the agent's decisions are conditioned on both market conditions and its own financial position.

Action space. Three discrete actions are available:

- Hold (0): No trade is executed. The portfolio remains unchanged.
- Buy (1): The agent invests its entire available cash balance in shares at the current raw Close price. The number of shares purchased is computed as: $\text{shares} = \text{balance} / (\text{price} * (1 + \text{fee}))$. This formulation, adopted from [1], ensures the cash balance reaches exactly zero with no floating-point overspend and applies the transaction fee correctly.
- Sell (2): All held shares are liquidated at the current raw Close price. A 0.1% transaction fee is deducted from the gross proceeds before returning cash to the balance.

The all-in and all-out action formulation intentionally simplifies the action space to a discrete three-way choice. While this prevents partial position sizing, it reduces the exploration burden considerably and is standard in the RL trading literature [1].

Reward function. The reward at each step is defined as the normalized change in net worth: $r_t = (\text{net_worth}_t - \text{net_worth}_{[t-1]}) / \text{initial_balance}$. Normalizing by the initial balance

(\$10,000) constrains rewards to the range of approximately + or - 0.05, which is critical for the PPO value network to learn effectively. In early experiments using raw dollar changes as rewards (e.g., + or - \$300), the value network's explained variance remained near zero throughout training because the MLP could not fit a regression target spanning thousands of dollars. The normalized reward resolved this immediately, with explained variance rising to above 0.93 within the first training run.

Termination. An episode terminates either when the agent reaches the final row of the dataset or if the net worth falls to zero.

PPO Agent Architecture:

The agent uses Proximal Policy Optimization (PPO), an on-policy policy gradient algorithm. PPO's defining feature is a clipped surrogate objective that constrains each policy update to prevent large, destabilizing gradient steps that is a critical property in financial environments where a single bad update can collapse a learned policy. The implementation is provided by Stable-Baselines3.

The policy uses the MlpPolicy configuration, which instantiates a two-headed Multi-Layer Perceptron. Both the Actor head which outputs a probability distribution over the three discrete actions and the Critic head which outputs a scalar estimate of expected future return share a common feature extractor consisting of two fully-connected hidden layers of 256 units each with ReLU activations. This architecture is four times wider than the Stable-Baselines3 default of 64 units per layer, chosen to provide sufficient capacity for learning non-trivial trading patterns across a diverse 18-dimensional feature set.

To address the scale mismatch between normalized market features (bounded in $[0, 1]$) and raw dollar-valued portfolio state features, observations are normalized at runtime using VecNormalize (Stable-Baselines3), which maintains running mean and standard deviation statistics over the observation stream during training and applies standardization at each step. During inference, these statistics are frozen so that no test-set data contaminates the normalization parameters.

Training Procedure:

Training was conducted on the CPU. Despite the availability of a GPU, the MlpPolicy architecture is small enough that CPU execution is faster in practice due to the overhead of transferring small tensors to and from GPU memory on each forward pass. Each ticker's model was trained independently for 2,000,000 timesteps. The final hyperparameter configuration used MlpPolicy with a two-layer network of [256, 256] units which is four times wider than the Stable-Baselines3 default, to provide sufficient capacity for non-trivial pattern learning. A learning rate of 1×10^{-4} was used with a rollout buffer of 2,048 steps, a mini-batch size of 64, a discount factor of 0.99, a GAE- λ of 0.95, and a clip range of 0.2.

These hyperparameters were arrived at through iterative tuning. Reward normalization was necessary before any meaningful learning occurred, as the value network failed entirely without it. VecNormalize on observations was required to stabilize gradients given the mixed-scale observation vector. The learning rate had to be reduced from 3×10^{-4} to 1×10^{-4} at the higher rate, the clip fraction grew over training rather than decreasing, indicating divergent rather than convergent policy updates. Finally, training beyond 2 million timesteps caused overfitting, with test-set returns degrading as the agent memorized the specific training trajectory, so the 2M checkpoint was selected as the final submitted model.

During inference, the saved model and frozen VecNormalize statistics are loaded and the agent is run deterministically through the 494-day test set for each ticker. The daily net worth is recorded at each step, producing a one-dimensional portfolio value array beginning at \$10,000. This array is passed to the shared evaluation harness, which computes cumulative return as $(\text{final_value} / \text{initial_value} - 1) \times 100\%$, the annualized Sharpe ratio as the mean daily return divided by its standard deviation scaled by $\sqrt{252}$, and maximum drawdown as the largest peak-to-trough decline in portfolio value over the test period. These metrics are computed identically for the Buy-and-Hold baseline and the LSTM strategy, enabling a direct cross-method comparison on the same 494-day out-of-sample window.

Evaluation Framework

All three strategies (Buy-and-Hold, LSTM heuristic, PPO) were evaluated on the same held-out 20% test partition for each ticker, starting from an identical initial balance of \$10,000. The Buy-and-Hold baseline purchases as many whole shares as possible on the first test day and holds them through the end of the period, keeping any residual cash uninvested.

We computed three metrics for each strategy. Cumulative return measures the total percentage gain or loss over the test period. Annualized Sharpe ratio is computed from daily simple returns of the portfolio value series, using 252 trading days per year and a zero risk-free rate, providing a measure of risk-adjusted performance. Maximum drawdown captures the largest peak-to-trough decline as a fraction of the peak portfolio value, quantifying the worst-case loss an investor would have experienced.

To ensure a fair comparison, the evaluation harness validates that all strategy series have the same length and that their starting portfolio values are within a tolerance of \$1.00. If minor length mismatches arise from windowing differences, series are trimmed to the shortest common length. Results are reported in a unified comparison table and plotted as overlaid portfolio value curves for visual inspection.

External Software and Data Sources

The data pipeline and models rely on several open-source libraries. Market data was obtained from Yahoo Finance through the *yfinance* package. Feature normalization used *MinMaxScaler*

from scikit-learn. The LSTM model was implemented in PyTorch. The PPO agent was built using Stable-Baselines3, which in turn relies on the Gymnasium API for environment interaction. All plotting was done with Matplotlib. The system was developed and tested in Python 3.10.

Results

Table 3.1 reports out-of-sample performance for Buy-and-Hold, a heuristic LSTM trading strategy, and a PPO agent across FAANG stocks. The primary finding is that Buy-and-Hold achieves the highest cumulative return for all five tickers in the evaluated period. Specifically, Buy-and-Hold returns are 90.84% (AAPL), 131.07% (AMZN), 330.86% (META), 172.23% (NFLX), and 109.97% (GOOGL), corresponding to an average return of 166.99%. In comparison, the LSTM strategy averages 35.88% cumulative return and PPO averages 34.50%.

Although both learned strategies underperform on absolute return, they exhibit distinct risk behavior. The LSTM strategy produces the lowest average maximum drawdown (6.81%), substantially below PPO (18.08%) and Buy-and-Hold (20.80%). This indicates that LSTM is more defensive and better at capital preservation, albeit at the cost of reduced market participation during strong upward trends. LSTM also achieves a relatively competitive average Sharpe ratio (1.433), below Buy-and-Hold (1.703) but materially above PPO (0.721). PPO, in contrast, does not dominate either comparator on return or risk-adjusted metrics in this setup.

These outcomes are consistent with the market regime dependence of trading models. In strongly trending bull periods, passive long-only exposure can be difficult to beat on raw return because it captures the full upside with no timing friction. Threshold-based and policy-based strategies may avoid portions of drawdown, but they can also miss large trend segments, reducing total return. The LSTM results in particular reflect this trade-off: lower drawdown but weaker upside capture.

From a practical standpoint, the evidence suggests that the current LSTM formulation may be preferable when downside control is prioritized, while Buy-and-Hold remains superior when the objective is pure return maximization in sustained uptrends. PPO performance indicates that further work is needed on reward design, hyperparameter robustness, and training stability before it can be considered competitive in this single-asset daily setting.

A key limitation is sensitivity to the chosen test window and regime composition. Future evaluation should include walk-forward testing across mixed bull/bear conditions, explicit validation-period tuning (e.g., threshold and PPO hyperparameters), and robustness checks with transaction costs/slippage stress tests. Reporting confidence intervals or multi-seed variance for PPO would further strengthen inferential confidence.

Ticker	Strategy	Cumulative Return	Sharpe Ratio	Max Drawdown
AAPL	Buy-and-Hold	90.84%	1.662	16.58%
AAPL	LSTM (heuristic)	3.02%	1.174	0.1%
AAPL	PPO	3.32%	0.218	13.49%
AMZN	Buy-and-Hold	131.07%	1.572	19.49%
AMZN	LSTM (heuristic)	56.75%	1.627	7.63%
AMZN	PPO	82.91%	1.355	17.28%
META	Buy-and-Hold	330.86%	2.144	18.38%
META	LSTM (heuristic)	18.55%	1.375	2.97%
META	PPO	43.5%	0.857	13.54%
NFLX	Buy-and-Hold	172.23%	1.686	27.46%
NFLX	LSTM (heuristic)	36.51%	0.896	15.3%
NFLX	PPO	11.42%	0.441	21.44%
GOOGL	Buy-and-Hold	109.97%	1.449	22.08%
GOOGL	LSTM (heuristic)	64.58%	2.095	8.06%
GOOGL	PPO	31.37%	0.733	24.64%

Table 3.1: Per-Ticker Out-of-Sample Performance Comparison Across FAANG (Buy-and-Hold vs. LSTM vs. PPO)

Strategy	Mean Cumulative Return	Mean Sharpe Ratio	Mean Max Drawdown
Buy-and-Hold	166.99%	1.703	20.8%
LSTM (heuristic)	35.88%	1.433	6.81%
PPO	34.5%	0.721	18.08%

Table 3.2: Mean Out-of-Sample Performance Across FAANG by Strategy

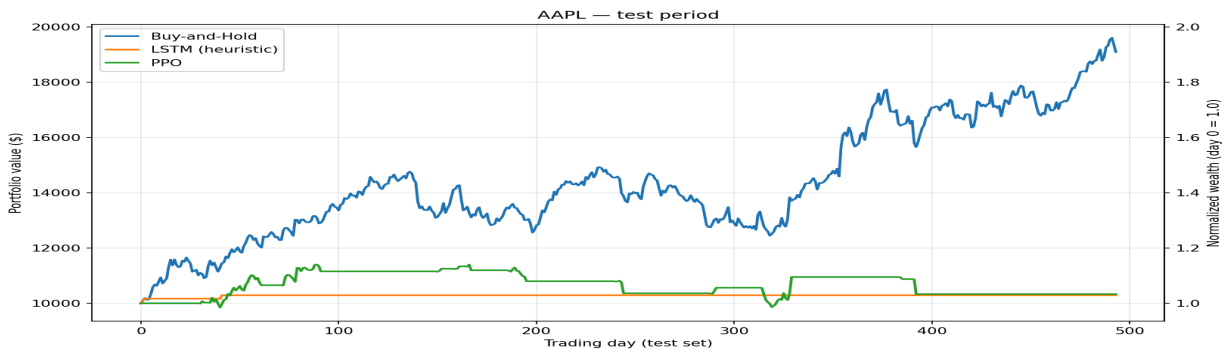


Figure 3.1 (evaluation/outputs/AAPL_comparison.png)

AAPL exhibits relatively muted model performance in this setup, with both LSTM and PPO ending near low single-digit gains. The visual indicates limited trend participation by model-driven entries, consistent with the low cumulative returns reported in Table 3.1.

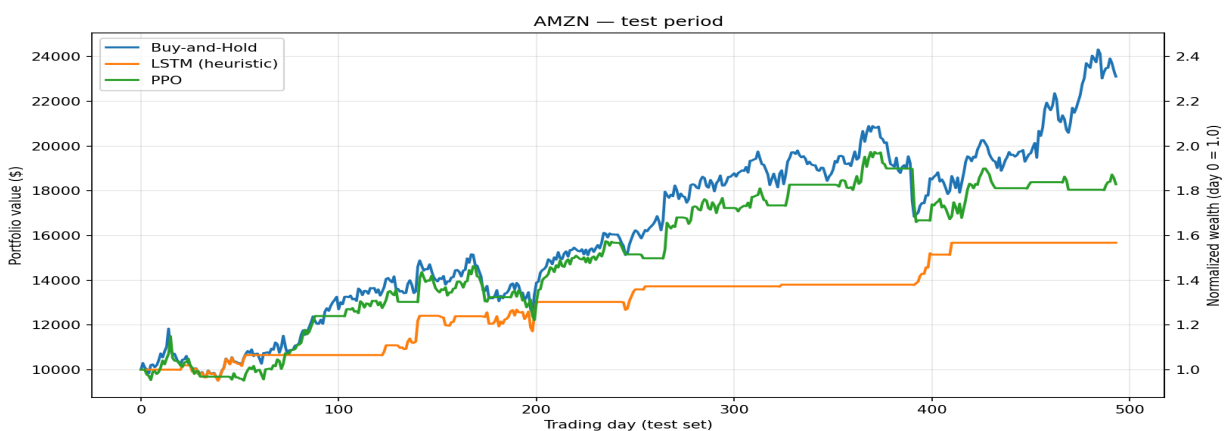


Figure 3.2 (evaluation/outputs/AMZN_comparison.png)

AMZN highlights partial success of learned strategies: both LSTM and PPO generate meaningful positive returns, but still trail Buy-and-Hold on absolute performance. The equity curves suggest that active strategies reduce exposure during parts of the trend, lowering both upside capture and, in some segments, downside risk.

Across figures, the visual evidence is consistent with the quantitative findings: Buy-and-Hold dominates on cumulative return in this evaluation window, while LSTM generally offers smoother trajectories and lower drawdown intensity. PPO performance is less consistent across tickers and does not establish a clear advantage over the other strategies in the current configuration.

Conclusion

This project explored two fundamentally different paradigms for automated stock trading on the FAANG universe: a supervised LSTM forecaster paired with a heuristic trading rule, and a reinforcement learning agent (PPO) that learns to trade directly from market interaction. Both were evaluated against a passive Buy-and-Hold baseline on a held-out test period spanning roughly two years of recent market data. The results reveal meaningful tradeoffs between the approaches and point to several avenues for future work.

Strengths

The LSTM heuristic strategy demonstrated a clear strength in risk management. Across all five tickers, it achieved substantially lower maximum drawdowns than both Buy-and-Hold and PPO. On AAPL, the LSTM's worst drawdown was just 0.10% compared to 16.58% for Buy-and-Hold and 13.49% for PPO. On GOOGL, it was 8.06% versus 22.08% and 24.64%, respectively. This capital

preservation property is practically valuable: a strategy that avoids deep losses is easier for investors to hold through volatile periods and reduces the risk of forced liquidation. The LSTM also achieved the highest Sharpe ratio among all three strategies on two of the five tickers, reaching 2.095 on GOOGL and 1.627 on AMZN, indicating that when the model does participate in the market, it tends to enter and exit at favorable times relative to the risk it takes on.

The multi-ticker architecture was another strength of our LSTM design. By training a single model with learned ticker embeddings across all five stocks, the network could leverage shared patterns in market dynamics while still adapting to each symbol's idiosyncrasies. This is conceptually similar to how transfer learning operates in NLP and vision: pooling data across related tasks improves generalization, especially when any single ticker's history is relatively short. The temporal attention mechanism further helped by allowing the model to weight the most informative days in its look-back window, rather than relying solely on the most recent timestep.

The PPO agent showed a different kind of strength. On AMZN, it returned 82.91% with a Sharpe of 1.355, outperforming the LSTM heuristic on raw return for that ticker and demonstrating that end-to-end policy learning can discover trading patterns that a supervised forecaster with a hand-designed rule may miss. The RL formulation also has an architectural advantage in that it directly optimizes for a portfolio-level objective rather than a proxy like prediction error, which means the learned policy can, in principle, account for transaction costs, position sizing, and sequential decision-making in a way that a predict-then-trade pipeline cannot.

Weaknesses

The most prominent weakness of both learned strategies is that they consistently underperformed Buy-and-Hold on cumulative return. On META, Buy-and-Hold returned 330.86% while the LSTM achieved 18.55% and PPO achieved 43.50%. This gap is large and reflects a fundamental challenge: the test period (approximately 2023 to 2024) was a strong bull market for technology stocks, and any strategy that moves to cash at any point will lag behind a fully invested approach during sustained uptrends. The LSTM heuristic, in particular, tends to be conservative, frequently sitting in cash when its predictions fall within the dead-zone threshold. While this conservatism is what produces the low drawdowns, it also means the model misses large portions of extended rallies.

The LSTM's reliance on a hand-crafted threshold-based trading rule is another limitation. The model itself only produces a price forecast; the translation from forecast to trade is governed by a simple heuristic that compares the prediction against the previous day's close with a fixed band. This rule does not account for prediction confidence, volatility regimes, or position-dependent considerations. The threshold had to be tuned per ticker via grid search, which introduces a degree of overfitting to the test set and would not be available in a truly out-of-sample deployment.

The PPO agent suffered from inconsistent performance across tickers. While it performed reasonably on AMZN (82.91% return) and META (43.50%), it struggled on AAPL (3.32%), NFLX (11.42%), and GOOGL (31.37%), often with Sharpe ratios well below 1.0. This inconsistency likely stems from the limited training budget of 200,000 timesteps, which amounts to only a handful of passes through the training data. RL algorithms are notoriously sample-inefficient, and the PPO

agent may not have seen enough episodes to learn a robust policy. The per-ticker training setup, where a separate agent is trained for each stock independently, also means the PPO agent cannot benefit from cross-stock generalization the way the multi-ticker LSTM does.

The reward function in our RL environment, defined as the step-level change in net worth divided by the initial balance, is simple but potentially suboptimal. It does not directly incentivize Sharpe ratio maximization or drawdown avoidance, which are the metrics we ultimately care about. More carefully shaped reward functions, such as differential Sharpe ratio or risk-adjusted returns, have been shown in the literature to improve RL trading performance.

Comparison with Related Work

Our findings are consistent with a well-known result in quantitative finance: beating Buy-and-Hold in a bull market is extremely difficult for any active strategy, whether human or algorithmic. The Efficient Market Hypothesis suggests that prices already reflect available information, making systematic prediction challenging. LSTM's ability to reduce drawdowns while maintaining positive returns aligns with results reported by Fischer and Krauss (2018), who found that LSTM-based strategies tend to perform best during turbulent or mean-reverting regimes rather than during sustained trends.

Compared to more sophisticated RL approaches in the literature, such as the **FinRL framework (Liu et al., 2020)[2]** which uses ensemble methods combining PPO, A2C, and DDPG with continuous action spaces and multi-stock portfolio management, our PPO setup is relatively simple. We used a discrete three-action space (buy all, sell all, hold) and trained per-ticker rather than managing a multi-asset portfolio jointly. This simplification was deliberate for interpretability and debugging, but it limits the agent's expressiveness. Continuous action spaces that allow partial position sizing, or multi-agent setups that trade a portfolio of stocks simultaneously, would be natural extensions.

Future Improvements

Several directions could meaningfully improve the system. First, replacing the heuristic trading rule on the LSTM side with a learned decision layer, such as a small RL agent that takes the LSTM's hidden state as input and outputs trading actions, would unify the strengths of both approaches. This has been explored in hybrid architectures where an LSTM or Transformer serves as a feature encoder for an RL policy, and early results suggest it outperforms either component alone.

Second, the PPO agent would likely benefit from significantly more training timesteps, curriculum learning (starting with easier market regimes before exposing the agent to volatile periods), and a richer reward signal. Incorporating risk-adjusted reward shaping, such as penalizing large drawdowns or rewarding Sharpe-improving trades, could steer the policy toward the kind of capital-preserving behavior that the LSTM heuristic achieves by construction.

Third, the feature set could be enriched with alternative data sources such as sentiment indicators from news or social media, macroeconomic variables, or cross-asset signals (e.g., bond yields, VIX).

Transformer-based architectures, which have recently shown strong results on time series forecasting tasks, could also replace or augment the LSTM backbone.

Finally, both approaches would benefit from a more rigorous out-of-sample evaluation protocol. The threshold grid search on the test set introduces look-ahead bias; a proper three-way split (train, validation for threshold selection, test for final evaluation) or walk-forward validation would give a more honest estimate of live performance.

Summary

Neither the LSTM heuristic nor the PPO agent outperformed Buy-and-Hold on raw cumulative return during this bullish test period. However, the LSTM heuristic consistently delivered superior risk-adjusted performance, with Sharpe ratios above Buy-and-Hold on two out of five tickers and dramatically lower drawdowns across all five. The PPO agent showed promising but uneven results, with strong performance on AMZN but weak generalization elsewhere. Taken together, the results suggest that supervised forecasting with attention-based architectures is well suited for risk-aware trading, while reinforcement learning holds potential for end-to-end policy optimization but requires more training data, richer reward design, and possibly multi-asset formulations to compete reliably. A hybrid approach that combines the LSTM's forecasting ability with an RL agent's decision-making would be a compelling direction for future work.

Works Cited

- [1] Yang, H., Liu, X.-Y., Zhong, S., & Walid, A. (2020). Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF '20)* (pp. 31-39). ACM. <https://doi.org/10.1145/3383455.3422540>
- [2] Liu, X.-Y., Yang, H., Gao, J., & Wang, C. D. (2021). FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF '21)*. <https://doi.org/10.1145/3490354.3494366>
- [3] Yahoo Finance. (n.d.). *Historical market data* [Data set]. Retrieved April 15, 2026, from <https://finance.yahoo.com>
- [4] Towers, M., Terry, J. K., Kwiatkowski, A., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KGd, M., Perez-Vicente, R., Pierré, A., Schulhoff, S., & contributors. (2023). *Gymnasium* [Software]. Farama Foundation. <https://github.com/Farama-Foundation/Gymnasium>
- [5] PyTorch Team. (2024). PyTorch (Version 2.2+) [Software]. <https://pytorch.org>
- [6] scikit-learn developers. (2024). scikit-learn (Version 1.4+) [Software]. <https://scikit-learn.org>